

LAB 09: Sualeh abbas, Burair Haidar

TASK 1:

Code:

```
int main(void)
{

    /* USER CODE BEGIN 1 */

    /* USER CODE END 1 */

    /* MCU Configuration-----*/

    /* Reset of all peripherals, Initializes the Flash interface and the Systick. */
    HAL_Init();

    /* USER CODE BEGIN Init */

    /* USER CODE END Init */

    /* Configure the system clock */
    SystemClock_Config();

    /* USER CODE BEGIN SysInit */

    /* USER CODE END SysInit */

    /* Initialize all configured peripherals */
    MX_GPIO_Init();
    MX_I2C1_Init();
    MX_USART2_UART_Init();
    /* USER CODE BEGIN 2 */
    uint8_t who_am_i = 0;
    char msg[64];

    // Small startup delay
    HAL_Delay(100);

    // Read the WHO_AM_I register from the LSM303AGR accelerometer
    HAL_StatusTypeDef status = HAL_I2C_Mem_Read(
        &hi2c1,          // I2C handle
        LSM303AGR_ACCEL_ADDR, // Device address (0x33 = read)
        LSM303AGR_WHO_AM_I_REG, // Register to read (0x0F)
        I2C_MEMADD_SIZE_8BIT, // Register address is 8-bit
        &who_am_i,          // Buffer to store result
        1,                 // Read 1 byte
        HAL_MAX_DELAY      // Timeout
    );

    // Check if communication succeeded
    if (status == HAL_OK)
    {
        snprintf(msg, sizeof(msg), "WHO_AM_I = 0x%02X\r\n", who_am_i);
        HAL_UART_Transmit(&huart2, (uint8_t*)msg, strlen(msg), HAL_MAX_DELAY);
        if (who_am_i == LSM303AGR_WHO_AM_I_VAL)
        {
            snprintf(msg, sizeof(msg), "SUCCESS: LSM303AGR detected!\r\n");
        }
        else
        {
            snprintf(msg, sizeof(msg), "WARNING: Unexpected WHO_AM_I value!\r\n");
        }
    }
}
```

```

    }
}
else
{
    snprintf(msg, sizeof(msg), "ERROR: I2C communication failed! Status = %d\r\n", status);
}

HAL_UART_Transmit(&huart2, (uint8_t*)msg, strlen(msg), HAL_MAX_DELAY); /* USER CODE END 2 */

/* Infinite loop */
/* USER CODE BEGIN WHILE */
/* USER CODE BEGIN WHILE */
while (1)
{
    /* USER CODE END WHILE */

    /* USER CODE BEGIN 3 */
    uint8_t who_am_i = 0;
    char msg[64];

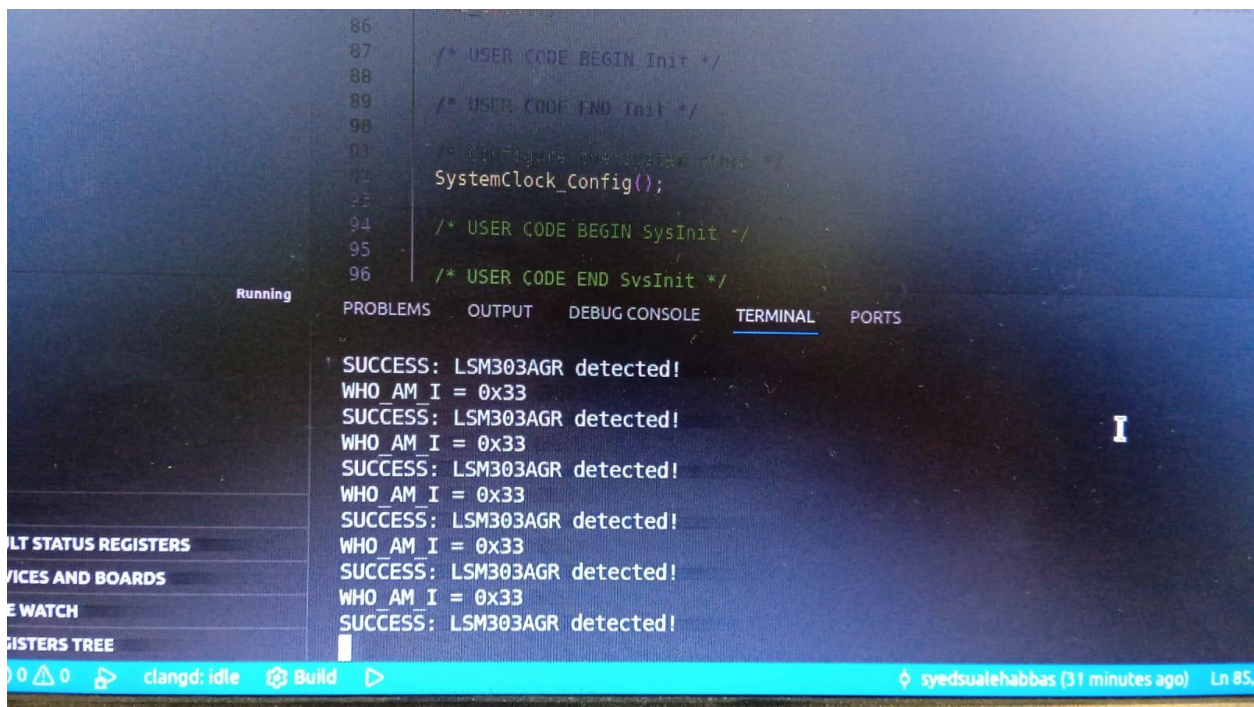
    HAL_StatusTypeDef status = HAL_I2C_Mem_Read(
        &hi2c1,
        LSM303AGR_ACCEL_ADDR,
        LSM303AGR_WHO_AM_I_REG,
        I2C_MEMADD_SIZE_8BIT,
        &who_am_i,
        1,
        HAL_MAX_DELAY
    );

    if (status == HAL_OK)
    {
        snprintf(msg, sizeof(msg), "WHO_AM_I = 0x%02X\r\n", who_am_i);
        HAL_UART_Transmit(&huart2, (uint8_t*)msg, strlen(msg), HAL_MAX_DELAY);
        if (who_am_i == LSM303AGR_WHO_AM_I_VAL)
            snprintf(msg, sizeof(msg), "SUCCESS: LSM303AGR detected!\r\n");
        else
            snprintf(msg, sizeof(msg), "WARNING: Unexpected WHO_AM_I value!\r\n");
    }
    else
    {
        snprintf(msg, sizeof(msg), "ERROR: I2C failed! Status = %d\r\n", status);
    }

    HAL_UART_Transmit(&huart2, (uint8_t*)msg, strlen(msg), HAL_MAX_DELAY);  HAL_Delay(1000);
}
/* USER CODE END 3 */
}

```

RESULT:



The screenshot shows an IDE with a code editor on the left and a terminal window on the right. The code editor displays C code for an LSM303AGR sensor. The terminal window shows the output of the program, which is a series of 'SUCCESS: LSM303AGR detected!' and 'WHO_AM_I = 0x33' messages. The IDE interface includes tabs for 'PROBLEMS', 'OUTPUT', 'DEBUG CONSOLE', 'TERMINAL', and 'PORTS'. The 'TERMINAL' tab is active, showing the program's output. The code in the editor is as follows:

```
86
87 /* USER CODE BEGIN Init */
88
89 /* USER CODE END Init */
90
91 /* Configure the system clock */
92 SystemClock_Config();
93
94 /* USER CODE BEGIN SysInit */
95
96 /* USER CODE END SysInit */
```

The terminal output is as follows:

```
SUCCESS: LSM303AGR detected!
WHO_AM_I = 0x33
SUCCESS: LSM303AGR detected!
WHO_AM_I = 0x33
SUCCESS: LSM303AGR detected!
WHO_AM_I = 0x33
SUCCESS: LSM303AGR detected!
WHO_AM_I = 0x33
SUCCESS: LSM303AGR detected!
WHO_AM_I = 0x33
SUCCESS: LSM303AGR detected!
WHO_AM_I = 0x33
```

The IDE status bar at the bottom shows 'clangd: idle', 'Build', and 'syedsualehabbas (31 minutes ago) Ln 85'.

TASK 2:

CODE:

```
void Init_LSM(void)
{
    uint8_t val;

    // Wake up + set ODR to 100Hz, enable X Y Z axes
    val = 0x67;
    HAL_I2C_Mem_Write(&hi2c1, LSM303AGR_ACCEL_ADDR_WRITE,
        CTRL_REG1_A, I2C_MEMADD_SIZE_8BIT,
        &val, 1, HAL_MAX_DELAY);

    // Normal mode, ±2g
    val = 0x00;
    HAL_I2C_Mem_Write(&hi2c1, LSM303AGR_ACCEL_ADDR_WRITE,
        CTRL_REG4_A, I2C_MEMADD_SIZE_8BIT,
        &val, 1, HAL_MAX_DELAY);

    HAL_Delay(10);
}

void Read_LSM(LSM_Data *data)
{
    uint8_t buf[6];

    // Read all 6 bytes at once using auto-increment (bit7 set)
    HAL_I2C_Mem_Read(&hi2c1, LSM303AGR_ACCEL_ADDR_READ,
        OUT_X_L_A, I2C_MEMADD_SIZE_8BIT,
        buf, 6, HAL_MAX_DELAY);

    // Combine high and low bytes
    data->raw_ax = (int16_t)((buf[1] << 8) | buf[0]);
}
```

```

data->raw_ay = (int16_t)((buf[3] << 8) | buf[2]);
data->raw_az = (int16_t)((buf[5] << 8) | buf[4]);

// Right-shift by 6 (10-bit left-justified), scale to g, subtract offset
data->ax = ((data->raw_ax >> 6) * ACC_SCALE / 1000.0f) - data->acc_offset_x;
data->ay = ((data->raw_ay >> 6) * ACC_SCALE / 1000.0f) - data->acc_offset_y;
data->az = ((data->raw_az >> 6) * ACC_SCALE / 1000.0f) - data->acc_offset_z;

// Tilt angles in degrees
data->angle_x = atan2f(data->ay, data->az) * RAD_TO_DEG;
data->angle_y = atan2f(-data->ax,
    sqrtf(data->ay * data->ay + data->az * data->az)) * RAD_TO_DEG;
}

void Offset_LSM(LSM_Data *data)
{
    float sum_x = 0, sum_y = 0, sum_z = 0;
    uint8_t buf[6];

    for (int i = 0; i < CALIB_SAMPLES; i++)
    {
        HAL_I2C_Mem_Read(&hi2c1, LSM303AGR_ACCEL_ADDR_READ,
            OUT_X_L_A, I2C_MEMADD_SIZE_8BIT,
            buf, 6, HAL_MAX_DELAY);

        int16_t rx = (int16_t)((buf[1] << 8) | buf[0]);
        int16_t ry = (int16_t)((buf[3] << 8) | buf[2]);
        int16_t rz = (int16_t)((buf[5] << 8) | buf[4]);

        // Right-shift by 6 before scaling
        sum_x += (rx >> 6) * ACC_SCALE / 1000.0f;
        sum_y += (ry >> 6) * ACC_SCALE / 1000.0f;
        sum_z += (rz >> 6) * ACC_SCALE / 1000.0f;

        HAL_Delay(10);
    }

    data->acc_offset_x = sum_x / CALIB_SAMPLES;
    data->acc_offset_y = sum_y / CALIB_SAMPLES;
    // Z: subtract 1g since gravity acts on Z when board is flat
    data->acc_offset_z = (sum_z / CALIB_SAMPLES) - 1.0f;
}

void Print_LSM(LSM_Data *data)
{
    char msg[100];

    // Multiply by 1000 to keep 3 decimal places, cast to int
    int ax = (int)(data->ax * 1000);
    int ay = (int)(data->ay * 1000);
    int az = (int)(data->az * 1000);
    int angleX = (int)(data->angle_x * 100);
    int angleY = (int)(data->angle_y * 100);

    snprintf(msg, sizeof(msg),
        "Ax:%d.%03d,Ay:%d.%03d,Az:%d.%03d,AngleX:%d.%02d,AngleY:%d.%02d\r\n",
        ax/1000, ax%1000 < 0 ? -(ax%1000) : ax%1000,
        ay/1000, ay%1000 < 0 ? -(ay%1000) : ay%1000,
        az/1000, az%1000 < 0 ? -(az%1000) : az%1000,
        angleX/100, angleX%100 < 0 ? -(angleX%100) : angleX%100,
        angleY/100, angleY%100 < 0 ? -(angleY%100) : angleY%100);

    HAL_UART_Transmit(&huart2, (uint8_t*)msg, strlen(msg), HAL_MAX_DELAY);
}

LSM_Data lsm = {0};

```

```

Init_LSM();

char cal_msg[] = "Calibrating... keep board flat!\r\n";
HAL_UART_Transmit(&huart2, (uint8_t*)cal_msg, strlen(cal_msg), HAL_MAX_DELAY);

Offset_LSM(&lsm);

char done_msg[] = "Calibration done!\r\n";
HAL_UART_Transmit(&huart2, (uint8_t*)done_msg, strlen(done_msg), HAL_MAX_DELAY);
// _____

/* USER CODE END 2 */

/* Infinite loop */
/* USER CODE BEGIN WHILE */
while (1)
{
if (HAL_GPIO_ReadPin(B1_GPIO_Port, B1_Pin) == GPIO_PIN_SET)
{
char cal_msg[] = "Calibrating...\r\n";
HAL_UART_Transmit(&huart2, (uint8_t*)cal_msg, strlen(cal_msg), HAL_MAX_DELAY);
Offset_LSM(&lsm);
char done_msg[] = "Done!\r\n";
HAL_UART_Transmit(&huart2, (uint8_t*)done_msg, strlen(done_msg), HAL_MAX_DELAY);
}
Read_LSM(&lsm);
Print_LSM(&lsm);
HAL_Delay(100);
// _____

}
/* USER CODE END 3 */
}

```

Results:

```

Ax:0.007,Ay:0.001,Az:0.988,AngleX:0.10,AngleY:0.42
Ax:0.007,Ay:0.009,Az:1.004,AngleX:0.56,AngleY:0.42
Ax:0.003,Ay:0.009,Az:0.980,AngleX:0.58,AngleY:0.20
Ax:0.007,Ay:0.009,Az:0.988,AngleX:0.57,AngleY:0.42
Ax:0.011,Ay:0.006,Az:0.992,AngleX:0.34,AngleY:0.65
Ax:0.003,Ay:0.006,Az:0.996,AngleX:0.34,AngleY:0.20
Ax:0.019,Ay:0.006,Az:0.976,AngleX:0.35,AngleY:1.12
Ax:0.019,Ay:0.009,Az:0.976,AngleX:0.58,AngleY:1.12
Ax:0.003,Ay:0.009,Az:0.984,AngleX:0.57,AngleY:0.20
Ax:0.012,Ay:0.006,Az:0.996,AngleX:0.34,AngleY:0.69
Ax:0.011,Ay:0.006,Az:0.984,AngleX:0.35,AngleY:0.65

```

EVALUATION QUESTION:

I2C and SPI are both communication protocols used to connect microcontrollers with peripherals, but they differ in speed, complexity, and usage. SPI is generally much faster, operating at speeds up to several MHz, which makes it suitable for applications where quick data transfer is required, such as displays and memory devices. On the other hand, I2C is slower, typically working at 100 kHz or 400 kHz, but it is more efficient in terms of wiring since it uses only two lines. In terms of complexity, SPI is simpler because it does not use addressing or acknowledgment; however, it requires a separate chip select line for each slave device. I2C is more complex as it uses addressing, acknowledgment, and supports multiple masters and slaves on the same bus, making it more flexible. In practice, SPI is preferred for high-speed applications, while I2C is commonly used for low-speed devices where saving pins and simplifying connections is more important.